

EXPERIMENT - 6

Title

Stored Procedure and Triggers for Employee Transfer and Salary Validation

Aim

To create a stored procedure `transfer_employee ( emp_id, new_dept_id)` with validation and error handling, and to implement triggers for salary validation and audit logging.

Objectives

1. To understand stored procedures in MySQL.
2. To create a procedure for transferring an employee to another department.
3. To validate employee and department details.
4. To implement error handling in a stored procedure.
5. To create a trigger for salary validation.
6. To create an audit trigger for recording salary changes.
7. To test different edge cases.

Software/ Tools Required

- ByteXL / Nimbus
- MySQL
- SQL

Theory

A **stored procedure** is a precompiled group of SQL statements stored in the database. It can be executed whenever required and can accept input parameters.

A stored procedure can contain conditions, validation, SQL statements, and error-handling mechanisms.

A **trigger** is a database object that automatically executes when a specified event occurs on a table. Triggers can be executed before or after operations such as **INSERT**, **UPDATE**, or **DELETE**.

Aryan Pratap Singh
25LBCS3266

In this experiment, a stored procedure is used to transfer an employee from one department to another. Triggers are used to validate employee salary and maintain an audit record when salary changes.

1. Stored Procedure

The `transfer_employee` procedure accepts an employee ID and a new department ID. It checks whether the employee and department exist before performing the transfer.

2. Salary Validation Trigger

A salary validation trigger checks the salary before an employee record is inserted or updated. It prevents invalid or negative salary values.

3. Audit Logging Trigger

An audit trigger automatically records salary changes in a separate audit table. It stores the employee ID, old salary, new salary, and time of change.

4. Error Handling

Error handling is used to display a meaningful message when an invalid employee ID or department ID is supplied.

SQL Implementation

1. Create Audit Table

```
CREATE TABLE SalaryAudit (  
    AuditID INT AUTO INCREMENT PRIMARY KEY,  
    EmpID INT NOT NULL,  
    OldSalary DECIMAL(10,2),  
    NewSalary DECIMAL(10,2),  
    ChangedAt TIMESTAMP DEFAULT CURRENT TIMESTAMP  
);
```

MySQL - db_454rqctf2

```

300 CREATE TABLE SalaryAudit (
301   AuditID INT AUTO_INCREMENT PRIMARY KEY,
302   EmpID INT NOT NULL,
303   OldSalary DECIMAL(10,2),
304   NewSalary DECIMAL(10,2),
305   ChangedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
306 );

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

2. Create Salary Validation Trigger

The following trigger prevents negative or zero salary values.

```
DELIMITER //
```

```

CREATE TRIGGER ValidateEmployeeSalary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE TEXT= 'Salary must be greater than zero';
    END IF;
END //

```

```
DELIMITER ;
```

MySQL - db_454rqctf2

```

308 DELIMITER //
309
310 CREATE TRIGGER ValidateEmployeeSalary
311 BEFORE INSERT ON Employee
312 FOR EACH ROW
313 BEGIN
314     IF NEW.Salary <= 0 THEN
315         SIGNAL SQLSTATE '45000'
316         SET MESSAGE TEXT = 'Salary must be greater than zero';
317     END IF;
318 END //
319
320 DELIMITER ;

```

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

3. Create Salary Validation Trigger for UPDATE

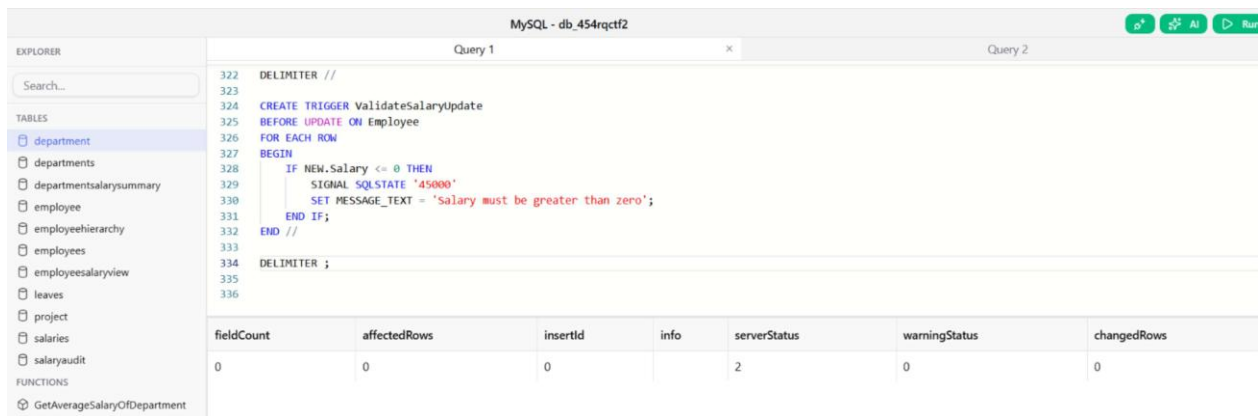
```
DELIMITER //
```

```

CREATE TRIGGER ValidateSalaryUpdate
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE TEXT= 'Salary must be greater than zero';
    END IF;
END//

DELIMITER;

```



4. Create Salary Audit Trigger

```

DELIMITER//

CREATE TRIGGER SalaryAuditTrigger
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO SalaryAudit
        (EmpID, OldSalary, NewSalary)
        VALUES
        (NEW.EmpID, OLD.Salary, NEW.Salary);
    END IF;
END//

DELIMITER;

```

EXPLORER

Search...

TABLES

department

departments

departmentsalarysummary

employee

employeehierarchy

employees

employeesalaryview

leaves

project

salaries

salaryaudit

337

338

339

340

341

342

343

344

345

346

347

348

349

350

CREATE TRIGGER salaryaudittrigger

AFTER UPDATE ON Employee

FOR EACH ROW

BEGIN

IF OLD.Salary <> NEW.Salary THEN

INSERT INTO salaryaudit

(EmpID, OldSalary, NewSalary)

VALUES

(NEW.EmpID, OLD.Salary, NEW.Salary);

END IF;

END //

DELIMITER ;

fieldCount	affectedRows	insertId	info	serverStatus	warningStatus	changedRows
0	0	0		2	0	0

5. Create Stored Procedure

DELIMITER//

```

CREATE PROCEDURE transfer_employee(
    IN emp_id INT,
    IN new_dept_id INT
)
BEGIN
    DECLARE emp_count INT DEFAULT 0;
    DECLARE dept_count INT DEFAULT 0;

    SELECT COUNT(*)
    INTO emp_count
    FROM Employee
    WHERE EmpID = emp_id;

    SELECT COUNT(*)
    INTO dept_count
    FROM Department
    WHERE DeptID = new_dept_id;

    IF emp_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Employee does not exist';

    ELSEIF dept_count = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE TEXT= 'Department does not exist';

    ELSE
        UPDATE Employee

```

```

        SET DeptID = new_dept_id
        WHERE EmpID = emp_id;
    END IF;
END//

DELIMITER;

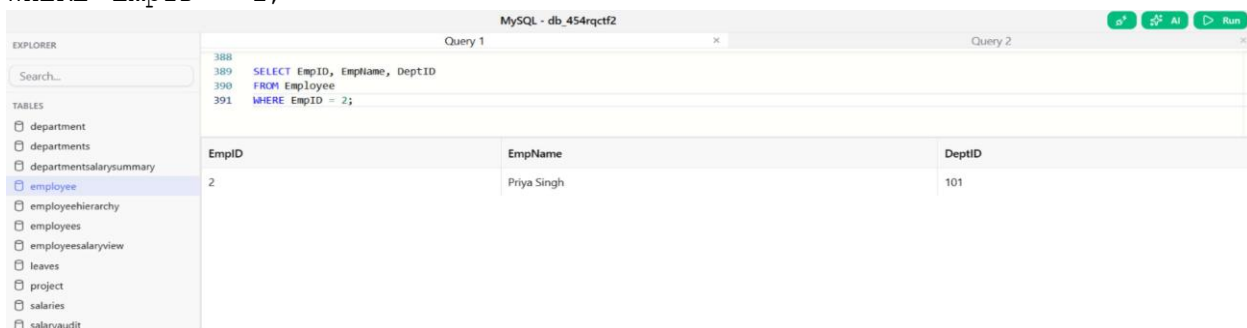
```

6. Display Employee Before Transfer

```

SELECT EmpID, EmpName, DeptID
FROM Employee
WHERE EmpID = 2;

```



EmpID	EmpName	DeptID
2	Priya Singh	101

7. Execute Employee Transfer

```
CALL transfer_employee(2, 103);
```

8. Verify Employee Transfer

```

SELECT EmpID, EmpName, DeptID
FROM Employee
WHERE EmpID = 2;

```



EmpID	EmpName	DeptID
2	Priya Singh	103

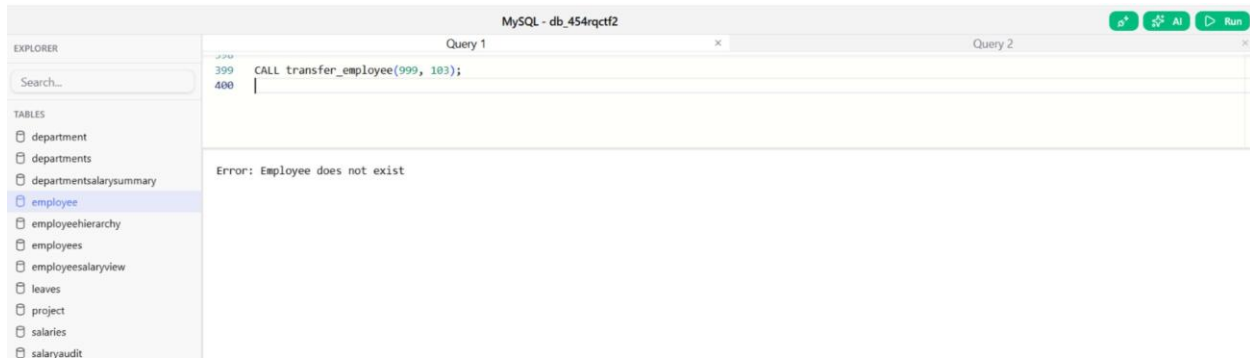
9. Test Invalid Employee

```
CALL transfer_employee(999, 103);
```

The procedure should generate an error message:

Aryan Pratap Singh
25LBCS3266

Employee does not exist



10. Test Invalid Department

`CALL transfer_employee(2, 999);`

The procedure should generate an error message:

Department does not exist



11. Test Salary Validation

Try inserting an employee with an invalid salary:

```
INSERT INTO Employee  
(EmpID, EmpName, Salary, DeptID, ProjectID)  
VALUES  
(101, 'Test Employee', -5000, 101, 201);
```

The trigger should reject the operation with:

Salary must be greater than zero



12. Test Salary Update Validation

```

UPDATE Employee
SET Salary= -1000
WHERE EmpID = 2;

```

The trigger should reject the update.

13. Test Salary Audit Logging

First change an employee's salary:

```

UPDATE Employee
SET Salary= Salary+ 2000
WHERE EmpID = 2;

```

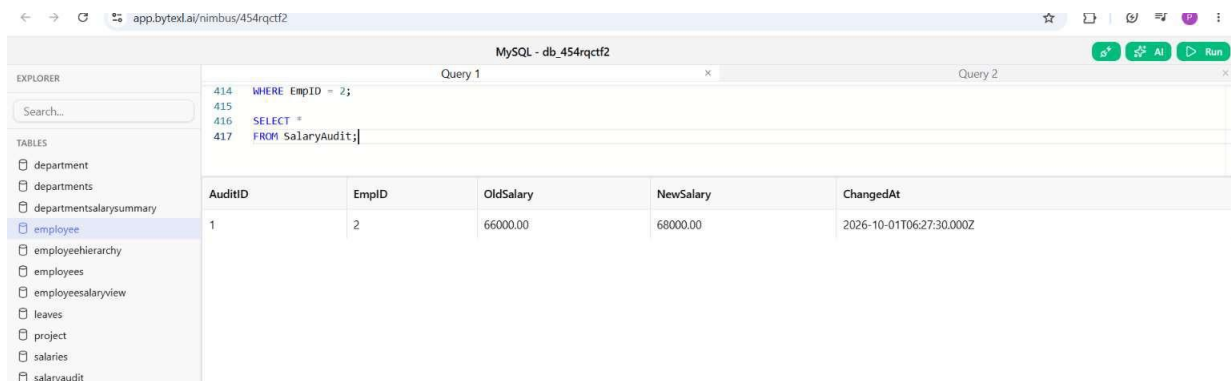
Then display the audit records:

```

SELECT*
FROM SalaryAudit;

```

The output should contain the employee ID, old salary, new salary, and change time.



Result

The stored procedure `transfer_employee` was successfully created with employee and department validation and error handling. Salary validation and salary audit logging triggers were also implemented and tested successfully.

Learning Outcomes

After completing this experiment, we are able to:

1. Create and execute stored procedures.
2. Use parameters in stored procedures.
3. Implement validation and error handling.
4. Create BEFORE and AFTER triggers.
5. Validate salary values using triggers.
6. Maintain an audit log for salary changes.
7. Test valid and invalid database operations.

Conclusion

This experiment demonstrated the use of stored procedures and triggers in MySQL. The employee transfer procedure validated employee and department details, while triggers were used for salary validation and audit logging. Different edge cases were tested successfully.